COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	17/07/2026	**DATE SUBMITTED:**	17/21/2026
NAME:	JAMES DANIEL VERANO	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 2 **Array, pointers, and Dynamic Allocation**

1. PROCEDURE OUTPUT

Screenshot

```cpp
#include <iostream>
#include <string.h>

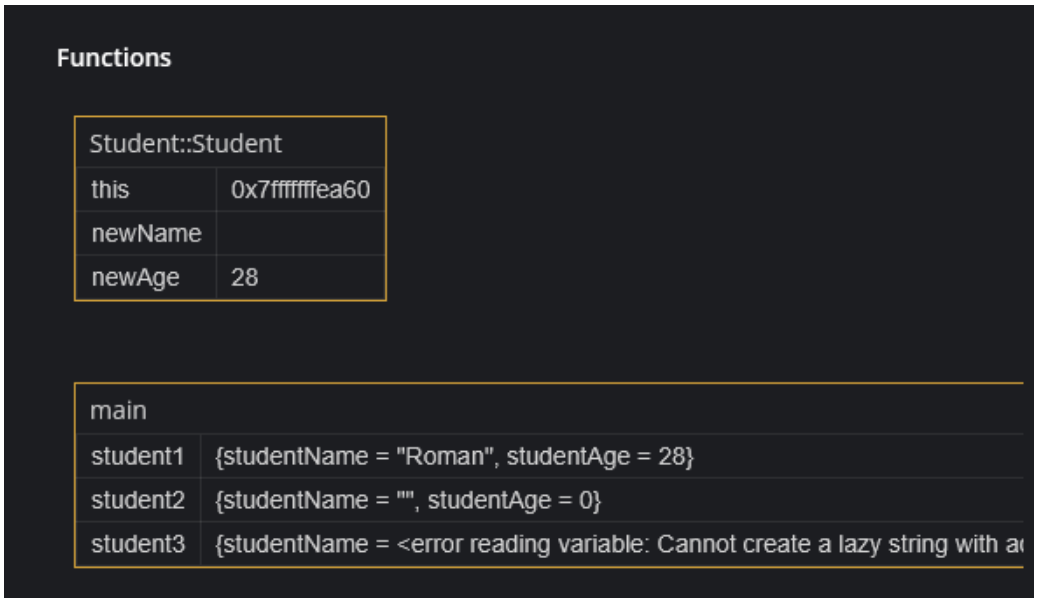
class Student {
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
    Student(std::string newName ="John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
        };
    ~Student(){
        std::cout << "Destructor Called." << std::endl;
    };

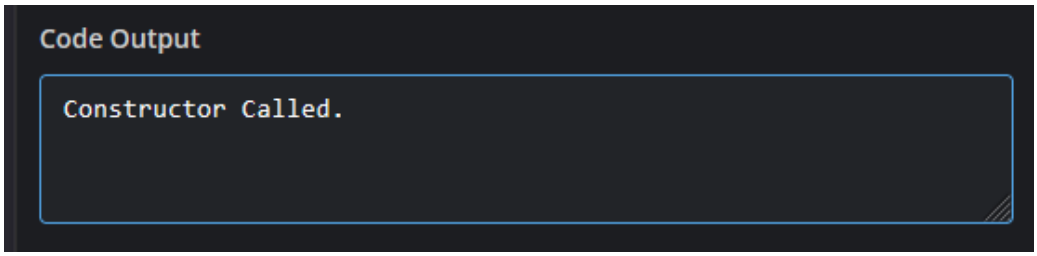
        //Copy Constructor
    Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    };

        //Display Attributes
    void printDetails(){
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    };
};
```

```
31
32  int main() {
33
34      Student student1("Roman", 28);
35      Student student2(student1);
36      Student student3;
37      student3 = student2;
38
39      return 0;
40  }
```

Observation





In the first student object instance, indicated as **Student student1("Roman", 28),** it initializes a function with the similar parameter condition. As the class accessed a certain that meets the condition's function parameter, **(std::string studentName, int studentAge)**, the method initialized the parameter's values to their respective class variable. As the visual represents, the constructor then constructs and initializes the object's member variables (**studentName** and **studentAge**). [**Line Code: 10 - 14]**

Functions

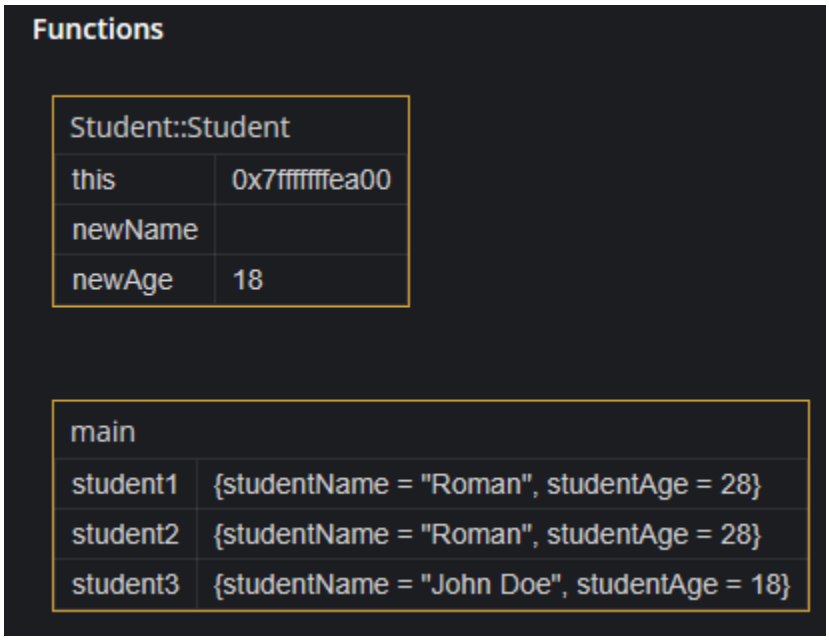
Student::Student	
this	0x7fffffffea30
copyStudent	@0x7fffffffea60

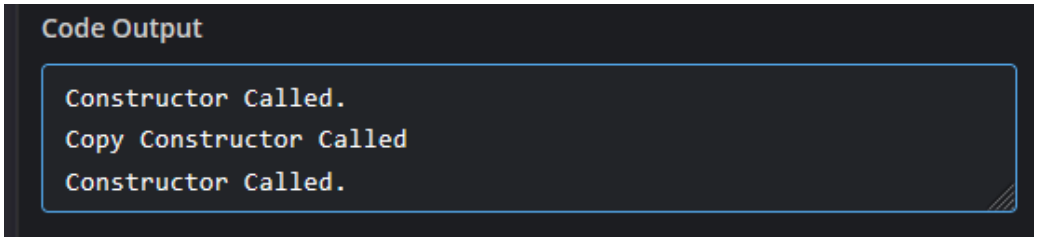
main	
student1	{studentName = "Roman", studentAge = 28}
student2	{studentName = "Roman", studentAge = 28}
student3	{studentName = <error reading variable: Cannot create a lazy string with a

Code Output

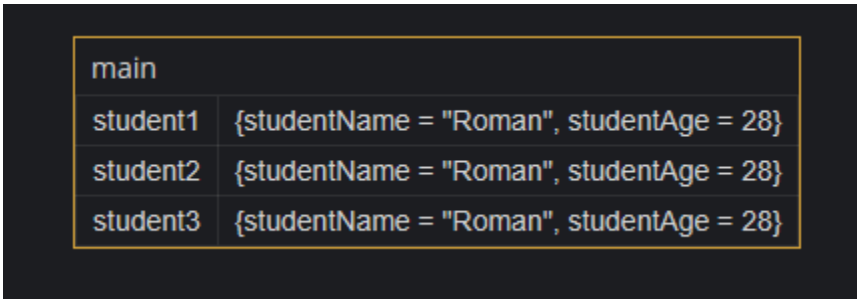
```
Constructor Called.
Copy Constructor Called
```

In the second student object instance, is the initialization of the copy constructor (**Line Code: 20 - 24**). As represented above, the copy constructor takes the reference to the entire **Student** object of the previous **Student instance** and initializes it into the new object that was created. Once initialized, the variable's value of student2, will have the same value that student1 has.





In the 3rd student object instance, **line code 36,** it called the constructor function without a given parameter. But where did the values come from? It came from the function's declared variable value that was inside the constructor caller **(Line Code: 10)**. The class accessed the Student3, gave the object the values initialized inside the function.



In this object instance, **Line Code 37**, the variable **student3** changed its member variables by initializing the variable into **student2**. Which overwrites the value of **student3**, making it all of them have similar name and age.

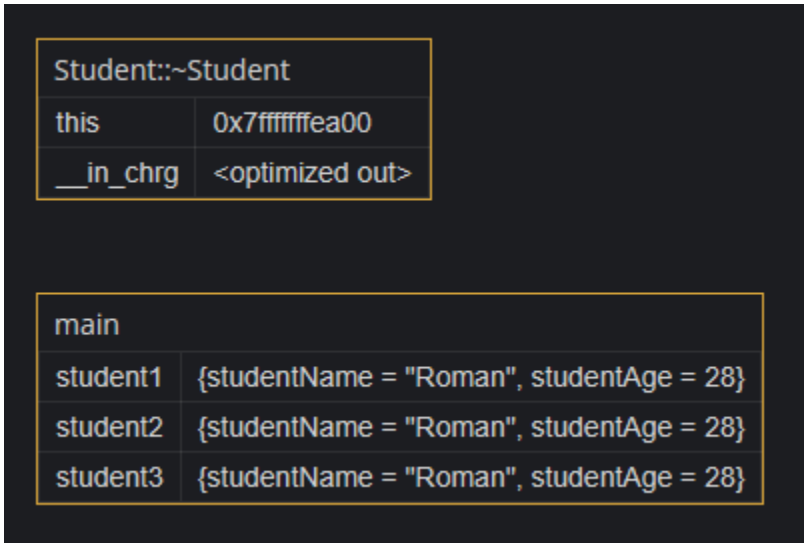
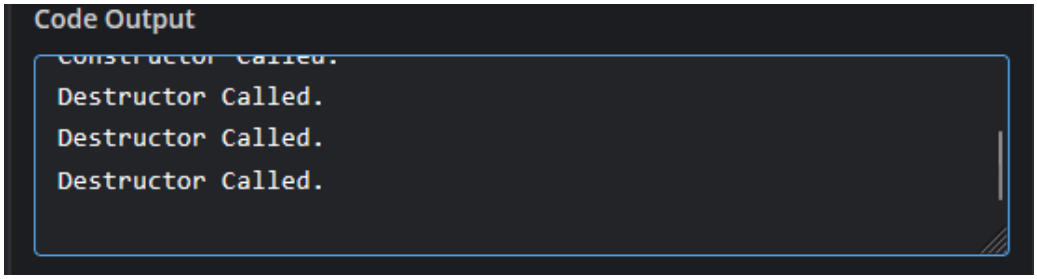
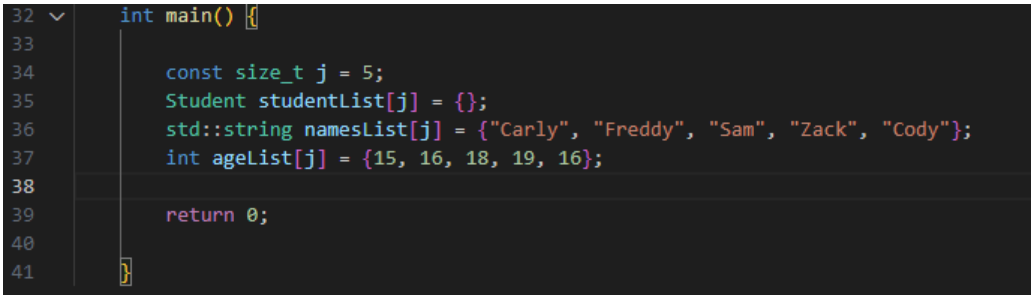
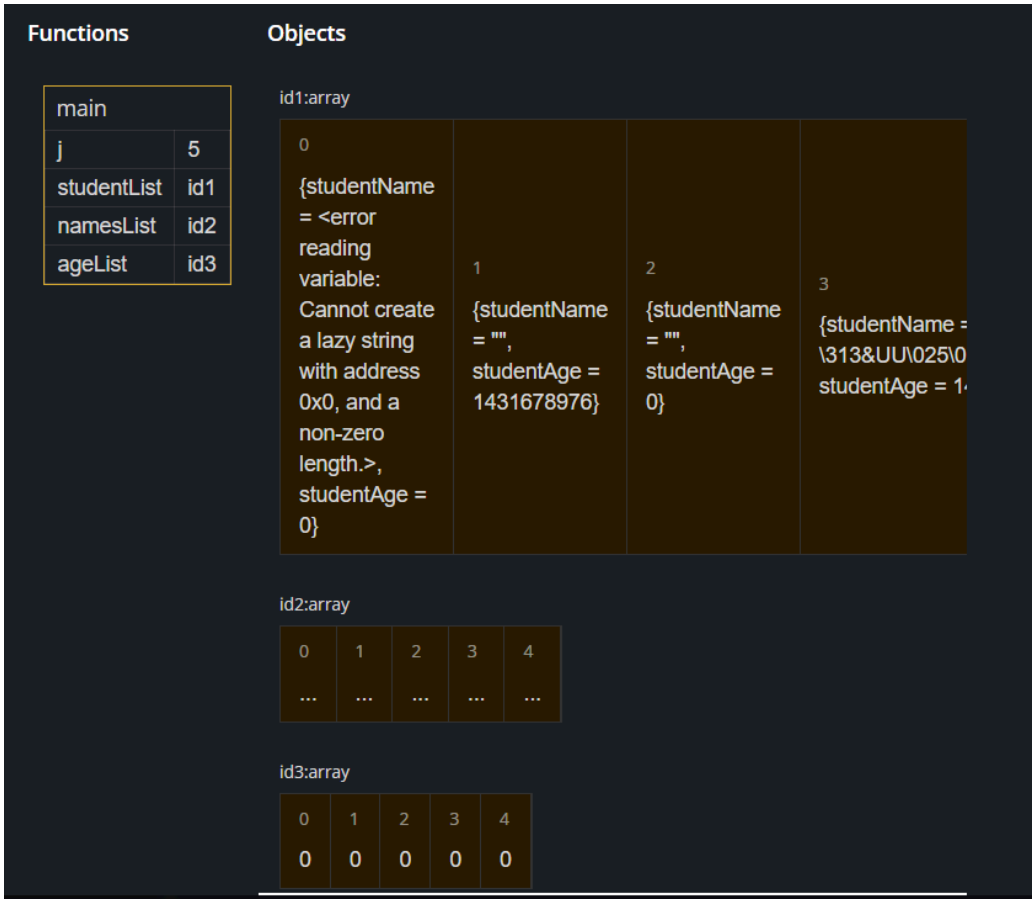
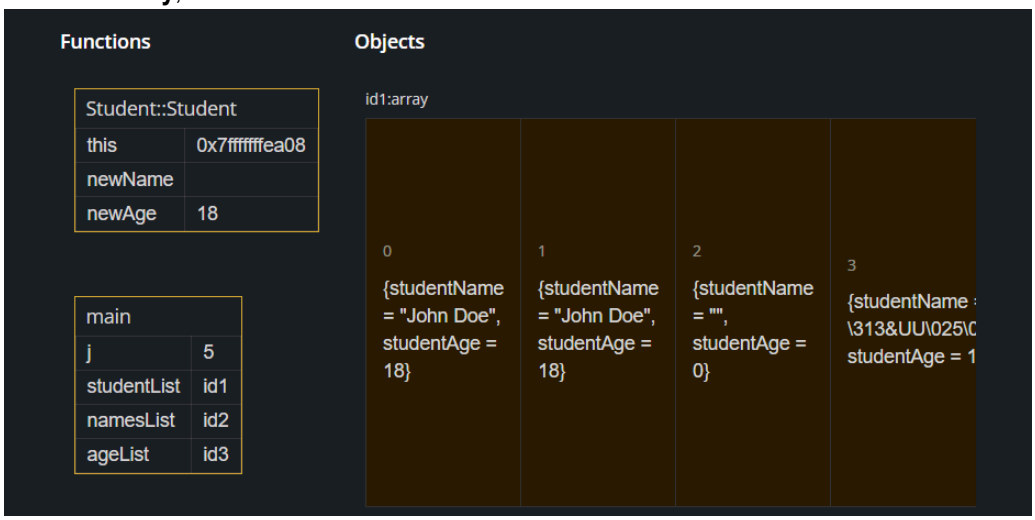
	  After the initialization of constructor and copy constructor, the next function that will be read in the class is the destructor (**Line Code: 15 - 17**). After its initialization, the destructor will clean up the object before it is destroyed. Just as the visual, the Student::~Student indicates the cleaning of the objects.

Table 2-1. Initial Driver Program

Screenshort	
Observation	



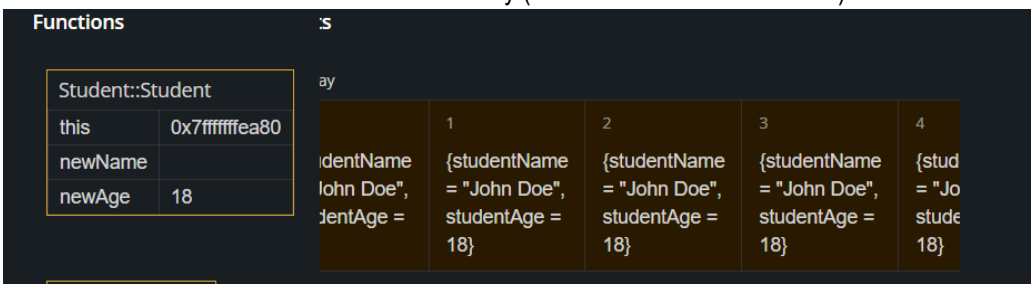
With the modified code, the main function initialized a constant size value 5 in an array. Indicated as **j = 5**. The modified code created 3 array objects with a constant size of 5, they are **studentList, nameList,** and **ageList**. In the flow of the program, the first that will be initiated will be the **id1:array**, which is the **studentList**.



Code Output

```
Constructor Called.
Constructor Called.
```

As the program reads through the first array type, the function needs to go through every index value and initialize the function. As it does not have any given parameter to input inside the array, it automatically invokes the default constructor for each element. It applies the **default parameter** values (**"John Doe"** and **18**) to each of the index values to be initialized inside the function to each of the object's member variables declared in the array. This processes the constructor function and the iteration of the array (**Line code: 10 - 14 & 34 - 35**).

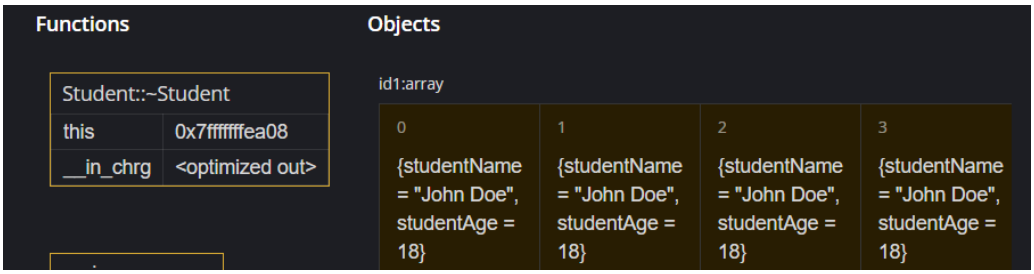


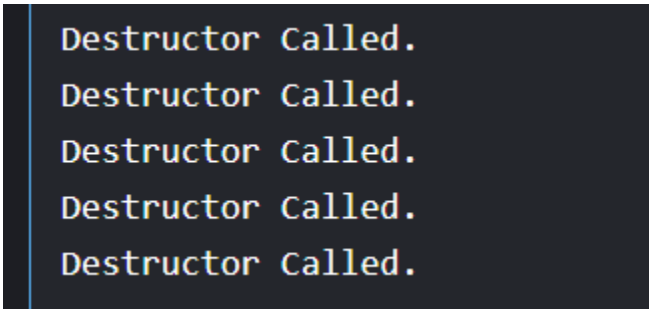

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
```

Here's the after product after the function iterates in the arrays and initialization of the object's member variable.



After the iteration and initialization of the first object named **studentList,** The program proceeds to initialize the next 2 array objects indicated **nameList** and **ageList**. Which is indicated above. This code instance is just a basic array storing and assigning index value to the given values. (**Line code: 36 - 37)** For example, first mentioned is "**Carly**" which the array assigns to **index 0**. The process also is just the same as the **ageList**.





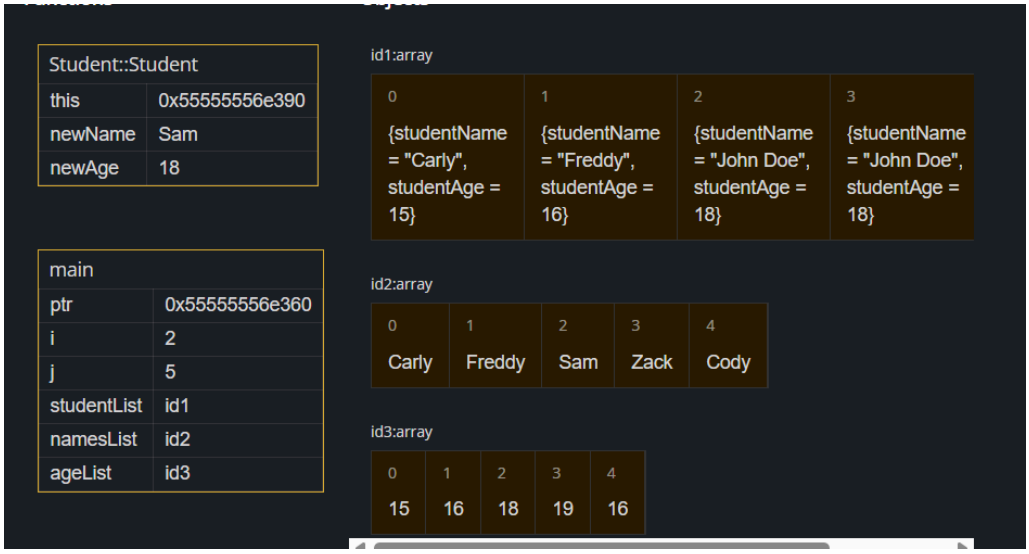
After the main function is initialized, the destructor activities to clear the object instance. It clears the values that are inside the constructed array, which is the array object named **studentList.** After the destructor is called, as you can see that the array still has values, it is what we call garbage value. To avoid it, we need to replace the values with either a value indicating non value, or null.

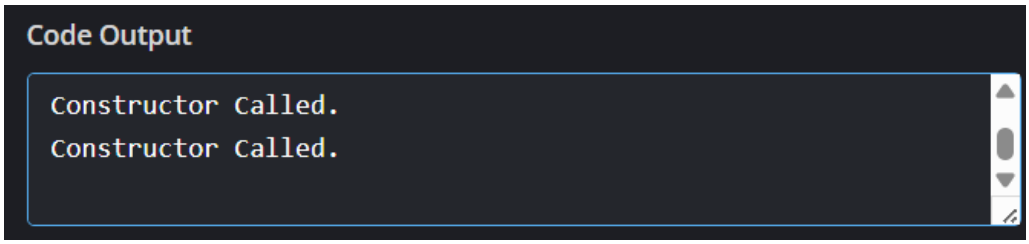
Table 2.2. Modified Driver Program with Student Lists

Loop A

```
40      for(int i = 0; i < j; i++){ //loop A
41      Student *ptr = new Student(namesList[i], ageList[i]);
42      studentList[i] = *ptr;
43      }
```

Observation





The loop A structure is the iteration process that is responsible for inputting the names that are inside the **nameList** inside the 1st array object, which is the **studentList**. By following the flow, if the **[i]** is less than the constant value **[j]**, the iteration will initiate the process of declaring a pointer creating new memory allocations. The concept of this constructor is to replace the given array value with the same index value assigned in the other object array as it is used as a parameter. Indicated as:

new Student(**nameList[i], ageList[i]**) = (“Carly”, 15)

This represents that it is assigning and storing the memory address of the newly created Student object. The next one is the initialization of the memory allocation. Where you will overwrite the value inside the object with the address inside the ***ptr**.

Loop B

```
for(int i = 0; i < j; i++){ //loop B
studentList[i].printDetails();
}
```

Observation

id1:array

0	1	2	3
{studentName = "Carly", studentAge = 15}	{studentName = "Freddy", studentAge = 16}	{studentName = "Sam", studentAge = 18}	{studentName = "Zack", studentAge = 19}

Student::printDetails	
this	0x7fffffffe9f0

main	
i	2
j	5
studentList	id1
namesList	id2
ageList	id3

In this loop iteration, is the printing of the newly created object after being initialized and allocated with the 2 object arrays. This iteration accesses the student object per index and it displays the value assigned to its index value. For example:

studentList[1], values that are in there are ("Freddy", 16)

Code Output

```
Carly 15
Freddy 16
```

Thus, it will print the value, Freddy 16.

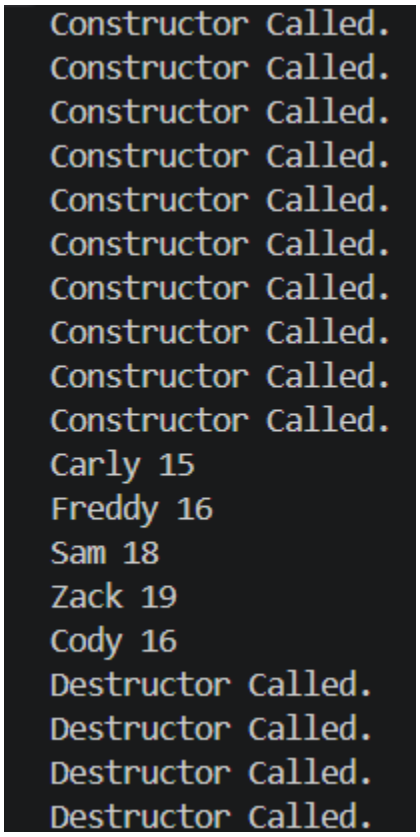
Output	
Observation	For the output, as you can see how the program constructed the overall iteration and how it initiated the destructor of objects after the initiations and printing. The overall code structure first constructed the memory allocations, initialization of functions, greetings of object instances and assigned index values. After all of the main function iteration, the main function initiated the cleaning of object instances.

Table 2-3. Final Driver Program

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

[Source Code]

```cpp
#include <iostream>
#include <string.h>

class Grocery {
    private:
        std::string name;
        double price;
        int quantity;
    public:
        Grocery(std::string itemName = "Unknown", double itemPrice = 0.0, int itemQuantity = 0)
            name = std::move(itemName);
            price = itemPrice;
            quantity = itemQuantity;
            std::cout << "Constructor Called." << std::endl;
        }
        virtual ~Grocery(){
            std::cout << "Destructor called." << std::endl;
        }
        Grocery(const Grocery &copyGrocery){
            std::cout << "Copy Constructor Called." << std::endl;\
            name = copyGrocery.name;
            price = copyGrocery.price;
            quantity = copyGrocery.quantity;
        }

        double calculateSum() const {
            return price * quantity;
        }

        std::string getName() const {
            return name;
        }

```

```
34          virtual void printDetails() const {
35              std::cout << "Name: " << this->name << " | Price: PHP " << this->price
36                        << " | Quantity: " << this ->quantity << " | Subtotal: PHP "
37                        << this->calculateSum() << std::endl;
38          }
39  };
40
41  class Fruit : public Grocery {
42      public:
43          Fruit(std::string itemName, double itemPrice, int itemQuantity)
44          : Grocery(itemName, itemPrice, itemQuantity) {}
45
46          ~Fruit() override {}
47
48          void printDetails() const override {
49              std::cout << "[Fruit]\t\t";
50              Grocery::printDetails();
51          }
52  };
53
54  class Vegetable : public Grocery {
55      public:
56      Vegetable(std::string itemName, double itemPrice, int itemQuantity)
57      : Grocery(itemName, itemPrice, itemQuantity) {}
58
59      ~Vegetable() override {}
60
61      void printDetails() const override {
62          std::cout << "[Vegetable]\t";
63          Grocery::printDetails();
64      }
65  };
66
```

```cpp
double totalSum(Grocery* list[], int size) {
    double total = 0;
    for (int i = 0; i < size; i++) {
        if(list[i] != nullptr) {
            total += list[i]->calculateSum();
        }
    }
    return total;
};

int main() {

    const size_t j = 4;

    Grocery* groceryList[j] = {nullptr};

    std::string nameList[j] = {"Apple", "Banana", "Brocolli", "Lettuce"};
    double priceList[j] = {10.0, 10.0, 60.0, 50.0};
    int quantityList[j] = {7, 8, 12, 10};

    for (int i = 0; i < j; i++) {
        if (i < 2) {
            groceryList[i] = new Fruit(nameList[i], priceList[i], quantityList[i]);
        } else {
            groceryList[i] = new Vegetable(nameList[i], priceList[i], quantityList[i]);
        }
    }

    std::cout << "\n\t\t|--- Jenna's Grocery List ---|\n" << std::endl;
    for (int i = 0; i < j; i++){
        groceryList[i]->printDetails();
    }

    double totalCost = totalSum(groceryList, j);
    std::cout << "\n Total Amount to Pay: PHP " << totalCost << "\n\n";
```

```
102
103
104     std::cout << "--- Problem 4: Deleting Lettuce ---" << std::endl;
105     for(int i = 0; i < j; i++) {
106         if(groceryList[i] != nullptr && groceryList[i]->getName() == "Lettuce") {
107             delete groceryList[i];
108             groceryList[i] = nullptr;
109             break;
110         }
111     }
112
113     for (int i = 0; i < j; i++) {
114         if (groceryList[i] != nullptr) {
115             groceryList[i]-> printDetails();
116         }else {
117             std::cout << "[Empty Slot] Element at index " << i << "was deleted." <<std::endl;
118         }
119     }
120
121     for (int i = 0; i < j; i++) {
122         delete groceryList[i];
123     }
124
125     return 0;
126 }
```

[Output]

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.

               |--- Jenna's Grocery List ---|

[Fruit]         Name: Apple | Price: PHP 10 | Quantity: 7 | Subtotal: PHP 70
[Fruit]         Name: Banana | Price: PHP 10 | Quantity: 8 | Subtotal: PHP 80
[Vegetable]     Name: Brocolli | Price: PHP 60 | Quantity: 12 | Subtotal: PHP 720
[Vegetable]     Name: Lettuce | Price: PHP 50 | Quantity: 10 | Subtotal: PHP 500

 Total Amount to Pay: PHP 1370

--- Problem 4: Deleting Lettuce ---
Destructor called.
[Fruit]         Name: Apple | Price: PHP 10 | Quantity: 7 | Subtotal: PHP 70
[Fruit]         Name: Banana | Price: PHP 10 | Quantity: 8 | Subtotal: PHP 80
[Vegetable]     Name: Brocolli | Price: PHP 60 | Quantity: 12 | Subtotal: PHP 720
[Empty Slot] Element at index 3was deleted.
Destructor called.
Destructor called.
Destructor called.
PS C:\Users\Striker\Documents\CPE21S4\CPE010\HOA 2>
```

[CODE ANALYSIS]

CODE ANALYSIS	
Objectives	**Procedure and Analysis**
Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes(such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.	For the first problem objective, we are tasked to create a class for the fruit and vegetable. Wherein, but classes contain a constructor, deconstructor, copy constructor, and copy assignment operator. They should also have all relevant attributes such as mentioned, and a calculated sum of all the product's quantity and price. Let's start introducing the implementation of my code above. First is, I initialized a **Grocery** class that consists of a private variable where we will be accessing the relevant attributes for the fruit and vegetable. In the public section, lays the structured form of constructor, deconstructor, copy constructor and copy assignment operator. (**Line Code: 9 - 24**) Additionally, inside the public section also lays the **calculateSum() function** (**Line Code: 26 -28**) to calculate for the total price of all the items combined in accordance to the quantity and price. Lastly in the public section, is a **printDetails() function** (**Line Code: 34 - 38**). This function initializes the printing of the data that are processed throughout the program. After the initializations and allocations, the **printDetails()** function will be the last to be executed excluding the deconstructor. Next, is the very objective. To create a Fruit and Vegetable class. According to my research, we can access a parent's class with another class through the inheritance and polymorphism way of constructing. First is we create a class named **Fruit** and **Vegetable** access the **Grocery** class to inherit the parameters and attributes of the parents class. With this, we will be able to execute another object instance. And to also be able to initialize the other execution such as the deconstructor, we used **virtual - override** to be able to **execute** the **inner function** first before the parents. This hides the complexity of the process of initialization and storing of data in another object instance, and relocating it to the Grocery list array in a way that it has its unique address towards the fruit and vegetable function. (**Line Code: 41 - 65**)
Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.	In the first objective, I mentioned that after being initialized and stored to another object instance created by the Fruit and Vegetable class, the Grocery class initializes these objects into an array list named **GroceryList**. Where it will contain all the items in Jenna's Grocery list where it can be accessed each saved instance that was processed by the classes. The display details about the item was already mentioned too, by iterating the **groceryList** array and using the **ptr**

	function printDetails() function that is the structure of displaying the final products and total price. (**Line Code: 91 - 97**)
Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.	The third objective is to create and initialize a function Totalsum that will calculate the sum of all objects listed in Jenna's Grocery List. As previously mentioned, we have created a **calculateSum() function**, now we are tasked to create a function where it calculates the total of the **whole list object instance** of all the grocery items and quantity. This is represented as in the **totalSum function**, where it accesses the **Grocery's class list[]** adds all price data stored in the **index value**, and each is initialized to update the value initialized to variable **totalCost**. (**Line Code: 67 - 75** and **100 - 101**)
Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.	After processing all the constructor and initializing classes, functions and all of the object instances, even in our procedural activity, we will be deleting a specific item list into the GroceryList and de-allocate the memory assigned. Which is represented to the function and procedure in the (**line code: 30 - 32** and **104 - 119**) In the first line code, is the taking of the name to be conditioned wherein, in the line **code 104 - 111**, is the deletion if inside the **groceryList[i]** consist of a value lettuce, thus, it deletes that list, and de-allocate the memory to **null**. For assurance, the **code 113 - 119** is only a newer version of the **printDetails() function** without the lettuce item.

CONCLUSION:

As I accomplished this Hands-on Activity, as I went through developing, integrating codes, analysis, debugging and creating the supplementary activity, I was able to further advance and gain a deeper understanding and knowledge about the basic fundamentals of advanced object-oriented programming. I was able to deeply understand how a constructor, copy constructor,deconstructor, and copy assignment operator works in a program. There are new terms that I have stumbled upon, which is the colon (**:**), that indicates "**is a type of**" meaning accessing the parent class in terms of a child class. **Virtual** for the child method to dynamically choose the correct method. And many more iterations and functions that are used to solve every single problem of the supplementary activity. As always, my knowledge is still unworthy enough of being able to competitively fight in programs, but learning more integration of basic fundamentals will further help me build up my structure.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google's AI Overview**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Terminologies, explanation of concept, code structure assistance and support.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I confirm that the code is from the basic integration of code assistance and knowledge from the basic structure of the procedure activity.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.